

LABORATORIO PRUEBAS UNITARIAS E INTEGRACIÓN (JAVA/SPRING BOOT)

CURSO: PRUEBAS DE SOFTWARE

OBJETIVO: Entender y afianzar sobre pruebas unitarias y de Integración mediante y la refactorización de código mediante el análisis estático con el uso de una herramienta IDE con un Lenguaje de programación y Framework robusto muy usado en la industria del Software.

SUGERENCIA: EVITAR el uso excesivo de herramientas de IA. Se sugiere un uso moderado para ciertos casos.

El caso de estudio es probar el servicio (API REST) del módulo de préstamos del “Banco Pascual Bravo”, en lo que se refiere a las pruebas Unitarias y de Integración usando IntelliJ Idea (Recomendada). El servicio está construido con Java/Spring Boot.

Asegurarse de que tenga instalada SonarQ y JDK en el equipo de trabajo. Descomprimir e importar el proyecto en el IDE IntelliJ.

1: Análisis Estático y Refactorización

- a. Ejecutar SonarQube (o SonarLint) para analizar la clase `PrestamoService.java`.
- b. Identificar las vulnerabilidades y Code Smells relacionados con:
 - Complejidad Cognitiva: Exceso de condicionales if/else anidados en el método `procesarSolicitud`.
 - Bucles Ineficientes: Uso de bucles `for` anticuados e innecesarios en `calcularDescuentoCartera`.
- c. Refactoriza el código

Observación: No dañar la lógica matemática del banco, solo mejorar la estructura.

2. Pruebas Unitarias (Aislamiento de la Capa de Servicio)

Para evitar tocar la base de datos central del banco durante el desarrollo, usarán dobles de prueba.

- a. Crear la clase `PrestamoServiceTest.java` en el paquete correspondiente a las pruebas (`src/test`)
- b. Utilizar JUnit y Mockito (`@Mock`, `@InjectMocks`) para probar las reglas de negocio en aislamiento.
- c. Diseñar casos de prueba que validen las políticas del Banco Pascual Bravo:
 - Rechazo automático de créditos superiores a \$50,000.
 - Tasa preferencial del 3.5% para clientes de la categoría "PREMIUM".
 - Tasa del 5.0% para clientes "REGULAR" y 6.0% para el resto.
- d. Excepciones de seguridad si los datos del cliente vienen nulos o con montos negativos.
- e. Alcanza al menos un 80% de cobertura de código.

3. Pruebas de Persistencia (H2 Database)

- a. Crear la clase `PrestamoRepositoryTest.java`.
- b. Utiliza `@DataJpaTest` para levantar una base de datos efímera en memoria.
- c. Inserta al menos 3 préstamos simulados utilizando `TestEntityManager`.
- d. Verifica que las consultas personalizadas del repositorio retornan los estados financieros correctos.

4. Pruebas de Integración (Simulación E2E de la API)

- a. Crea la clase `PrestamoControllerTest.java`.

- b. Utiliza `@SpringBootTest` y `MockMvc` para simular las peticiones HTTP que haría el frontend web del Banco Pascual Bravo.
- c. Valida que el endpoint `/api/prestamos/solicitar` responda con un código HTTP 200 OK y el JSON esperado ante solicitudes válidas. Usando el IDE IntelliJ.

Entregables:

Código fuente refactorizado cumpliendo los estándares de calidad (0 alertas críticas en SonarQube): Comprima el código en .zip

Suite de pruebas completa (`src/test/java`).

En un documento con portada. Reporte evidenciando con capturas de pantalla el cumplimiento de la cobertura de pruebas de al menos 80%. Y los casos de prueba. Anexar un Párrafo en donde el equipo de trabajo plasme las conclusiones de lo que hizo. Mencione, además, su opinión sobre este tipo de pruebas. Que tan importantes y útiles le parecen.

Observación: Utilice la ayuda de la IA cuando sea necesario.